

Starter Queries — Build Block

Copy these into `explore.py` and run them. Change the SQL to explore further.

How to use these: Open `explore.py` in the repo root. Replace the SQL inside the triple quotes with the query below. Save. Run:

Windows: `python explore.py` **Mac/Linux:** `python3 explore.py`

Each query has a "*Try changing...*" prompt. That's where the discovery happens — the Gotcha you need for your reference doc shows up when you modify the query and the number moves (or doesn't) in a way you didn't expect.

Attrition

Tables: `fct_separation`, `rpt_attrition_monthly`, `dim_worker_snapshot`

Query 1 — Look at the raw data

```
SELECT * FROM fct_separation LIMIT 5
```

Notice which columns exist. What types of workers are in here?

Query 2 — Who left in 2025?

```
SELECT worker_type, count(*) AS n
FROM fct_separation
WHERE year(separation_date) = 2025
GROUP BY worker_type
ORDER BY 1
```

Try changing: remove the worker_type filter entirely. How many total separations? Should all of them be in your attrition rate?

Query 3 — Count the Regular separations

```
SELECT count(*) AS separations_2025
FROM fct_separation
WHERE year(separation_date) = 2025
AND worker_type = 'Regular'
```

This is your numerator. Now you need a denominator — but from where?

Query 4 — Check the other attrition table

```
SELECT * FROM rpt_attrition_monthly
ORDER BY month DESC LIMIT 5
```

How many columns does this table have? Could you compute a rate from it alone?

Query 5 — Get a denominator (beginning of 2025)

```
SELECT count(DISTINCT employee_number) AS hc
FROM dim_worker_snapshot
WHERE snapshot_date = DATE '2024-12-31'
AND worker_type = 'Regular'
```

```
AND employment_status IN ('Active', 'On Leave')
```

Try changing: replace '2024-12-31' with '2025-12-31'. Compare the two numbers. Now divide your Query 3 separations by each. How far apart are the two rates?

That spread is your Gotcha. Same numerator, different denominators, materially different rates. Write it up.

Compensation

Table: fct_compensation

Query 1 — Look at the raw data

```
SELECT * FROM fct_compensation LIMIT 5
```

Notice the columns. What currency are the salary figures in?

Query 2 — The naive average

```
SELECT round(avg(base_salary_local)) AS avg_salary  
  
FROM fct_compensation
```

That's a number. Would you send it to your VP? What would you want to know first?

Query 3 — Break it out by currency

```
SELECT currency_code,  
  
       count(*) AS n,  
  
       round(avg(base_salary_local)) AS avg_salary  
  
FROM fct_compensation  
  
GROUP BY currency_code  
  
ORDER BY 1
```

Compare the per-currency averages to the blended average from Query 2. How different are they? Does the blended number mean anything?

Query 4 — Who's included?

```
SELECT w.worker_type,  
  
       count(*) AS n,  
  
       round(avg(c.base_salary_local)) AS avg_salary  
  
FROM fct_compensation c  
  
JOIN dim_worker w ON w.worker_id = c.worker_id  
  
WHERE c.currency_code = 'USD'  
  
GROUP BY w.worker_type  
  
ORDER BY 1
```

Try changing: remove the WHERE currency_code = 'USD' line. Does it matter?

Query 5 — Base salary vs total comp

```
SELECT round(avg(base_salary_local)) AS avg_base,  
  
       round(avg(base_salary_local * (1 + target_bonus_pct))) AS avg_total  
  
FROM fct_compensation  
  
WHERE currency_code = 'USD'
```

"What do we pay people?" — does that mean base, or base plus bonus? How big is the gap?

Your Gotcha is in Query 3 or Query 5 — either the currency mixing or the base-vs-total ambiguity. Write up whichever one surprised you more.

Engagement

Table: fct_engagement_survey

Query 1 — Look at the raw data

```
SELECT * FROM fct_engagement_survey LIMIT 10
```

Notice the columns. How many items per person? What's the last column?

Query 2 — The naive trend

```
SELECT year(wave_date) AS yr,  
  
       round(avg(response_value), 2) AS avg_score  
  
FROM fct_engagement_survey  
  
WHERE item_code IN ('ENG01', 'ENG02', 'ENG03')  
  
GROUP BY year(wave_date)  
  
ORDER BY 1
```

Look at the year-over-year change. Is there a jump that looks unusually large? What could explain it?

Query 3 — Add one column and look again

```
SELECT year(wave_date) AS yr,  
  
       scale_max,  
  
       count(*) AS n,  
  
       round(avg(response_value), 2) AS avg_score  
  
FROM fct_engagement_survey  
  
WHERE item_code IN ('ENG01', 'ENG02', 'ENG03')  
  
GROUP BY year(wave_date), scale_max  
  
ORDER BY 1
```

Compare this to Query 2. What changed between 2024 and 2025 besides the score?

Query 4 — Who responded?

```
SELECT w.worker_type, count(*) AS n  
  
FROM fct_engagement_survey e  
  
JOIN dim_worker w ON w.worker_id = e.worker_id  
  
GROUP BY w.worker_type  
  
ORDER BY 1
```

Your Define spec said Regular employees only. Can you filter that in the survey table without this join?

Your Gotcha is in Query 2 → Query 3. The jump that looked like improvement might be something else entirely. Write it up.

Remember: change the SQL inside the triple quotes in `explore.py`, save, run. The `read_only=True` in the script means you can't break the database — experiment freely. Every query on this page has been verified against the workshop warehouse.